

内置特征

一致的编码风格

风格一致有助于协同，也助于演进

使用编程规范来统一风格

规范体现了一致性，也体现了最佳实践

需要成文的规范

建立对规范的共识

保持规范的持续演进

基于日常协同来统一风格：代码评审和结对编程

有意义的命名

命名应该反映业务概念

从业务视角而不是开发视角对概念命名

向设计意图的表达优化：如流利接口

简洁的行为实现

越简短，越简单

保持一直的抽象层次

尽量降低代码复杂度

高内聚和低耦合的结构

内聚的本质是关注点分离

耦合必然存在，不良的耦合对设计有严重影响

从演进的角度考虑内聚性和耦合性

避免强耦合和有问题的耦合

没有重复

重复是最强的耦合

影响可理解性

影响可维护性

关注重复可以发现改进机会

重复中发现需要分离的关注点

没有多余的设计

“画蛇添足”意味着增加理解和维护成本

避免刻板印象的遵循设计范例：注重实效

及时删除不再有用的代码

避免为不可预见的未来编写代码：简单设计

具备自动化测试

自动化测试很重要

功能正确性的保证

提升设计的可理解性

保障系统的持续演进

不仅仅是质量保证，测试的本质是契约，也是资产的说明